Lab 0: ESP32 Microprocessor and Robot Analysis

Objectives

1. Setup the ESP32-S3 microprocessor
2. Analyse robot designs from previous competitions

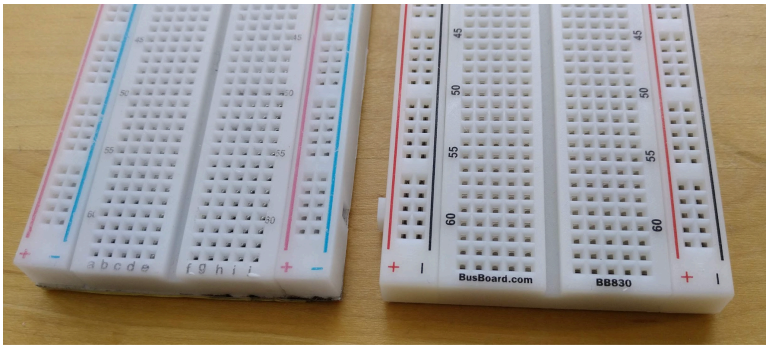
Tasks

1. Practice soldering header pins
2. Set up an esp32-s3 on breadboard
3. Download example code to ESP32
4. Set up and test the display
5. Analyse robot designs

Materials

- 1x ESP32 board per bench (= two per team)
- 1x USB cable
- 1x USB C adapter as needed

One Thing About Breadboards…



The "BusBoard.com" branded board is your premium breadboard for this course. It has high quality spring contacts that are more reliable than the contacts in the more common standard quality boards.

We have one BusBoard per table for ENPH 253, and all of you should have one more, left over from ENPH 259.

References

[Website] ESP32 hardware reference
[Github] ESP32-S3 Example Code

[YouTube] How To Solder The Header Pins To The BluePill Board (Note: we switched from the BluePill to the ESP32 but the technique for soldering a long row of headers might be useful for the work in this lab)
and in case you haven't soldered before:
[YouTube] Soldering Workshop
[YouTube] Generic Soldering Info and Health Hazards

Pre-Lab

Logbooks

Start a Logbook in which you keep detailed records of your learning journey during the course. This can include links, screenshots, notes on videos you watched, websites and blogs you visited, code snippets you've written, circuit diagrams and layouts you used, troubleshooting techniques you developed and on and on. We usually use a Google Doc for our Logbooks but any document format you find useful works. Aim to create structure to the deluge of information you are trying to capture (i.e. use headings and internal links cross referencing your understanding). It won't be perfect but you should push back against the entropic march of reality.

A template for log books is here (we provide this exact template to the capstone project teams).

Soldering tutorials

Watch the How To Solder The Header Pins To The BluePill Board video to see some technique involving soldering a long row of headers (and Soldering Workshop if you need a refresher, or have not done any soldering before).

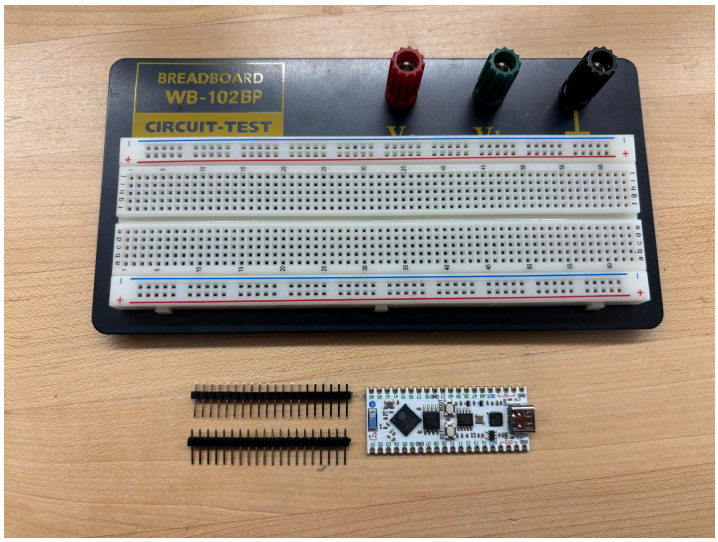
Lab 0

Soldering Practice

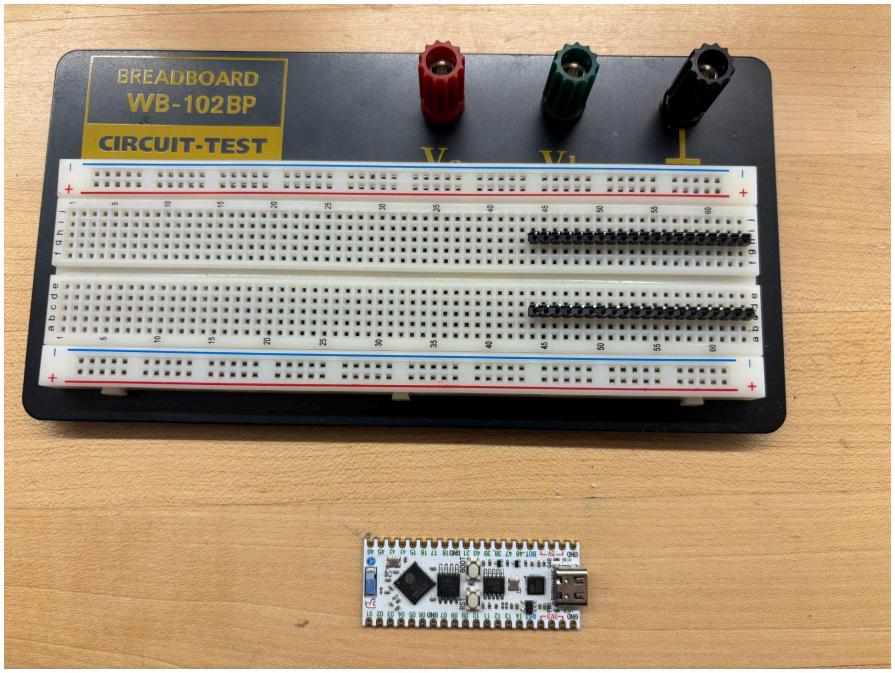
Please do not solder your ESP32 directly into anything. We would like to reuse as many ESP32 boards as we can next year. Instead, to interface your ESP32 with your soldered components later in the course, please use male header pins.

Soldering practice steps:

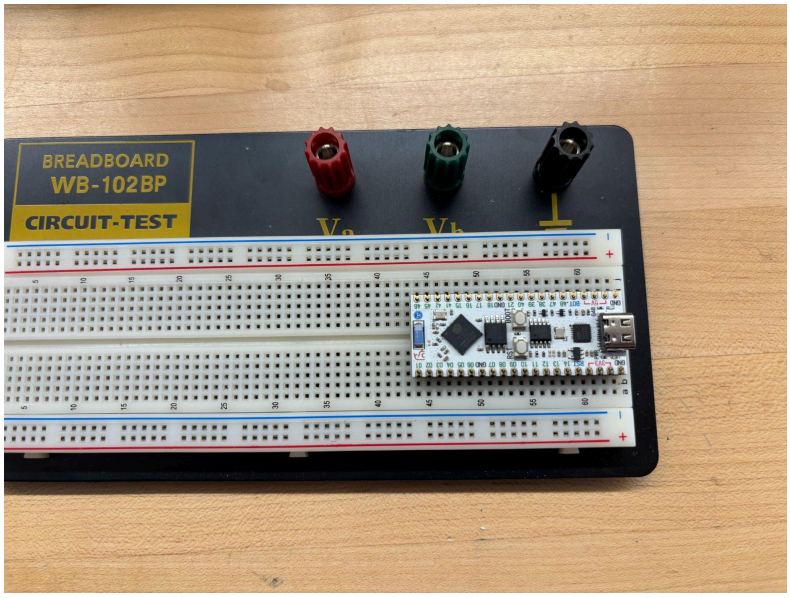
1. You will need the following components:
   - An ESP32-s3
   - 2 rows of header pins (come with a new esp32 and we have extra if needed)
   - A breadboard



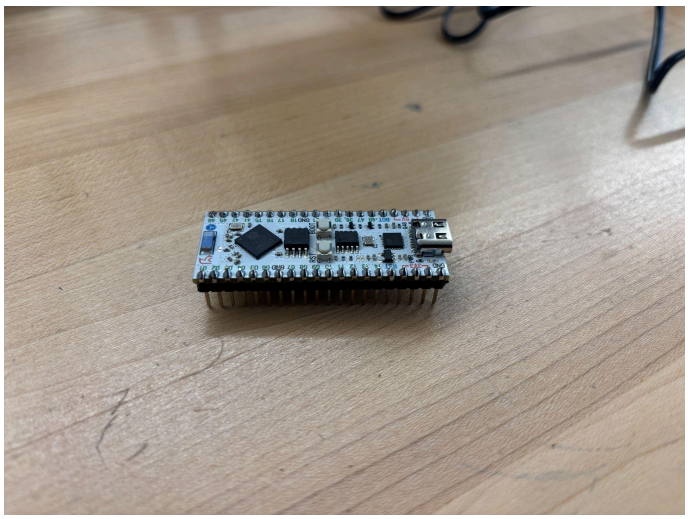
2. Place the header pins in the breadboard, aligned such that the esp32 could be placed with the pins inserted into each hole of the devboard



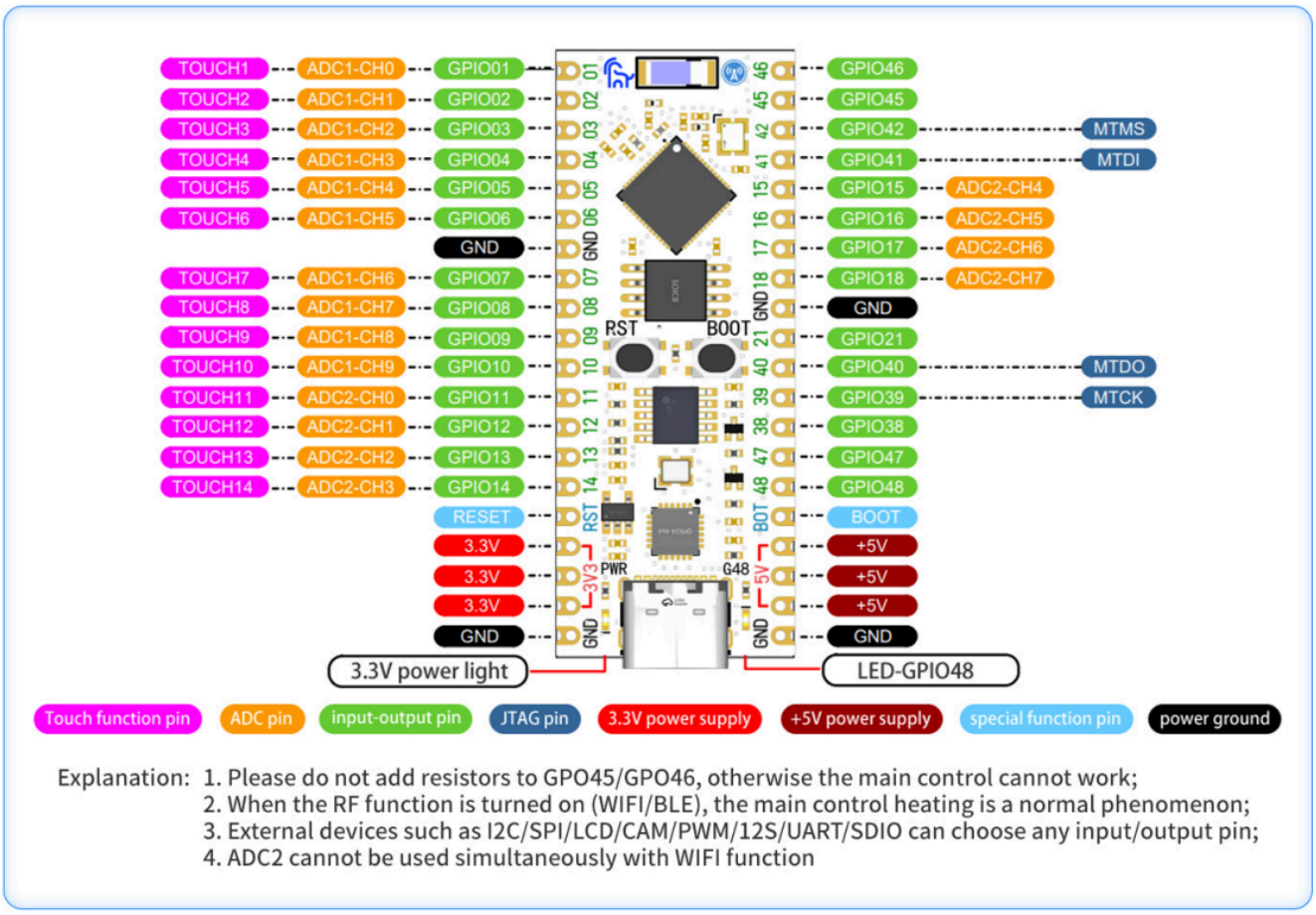
3. Place the esp32 devboard on the header pins such that each pin goes through a devboard pin hole



4. Solder each pin



ESP32-S3 Devkit kit pinout:

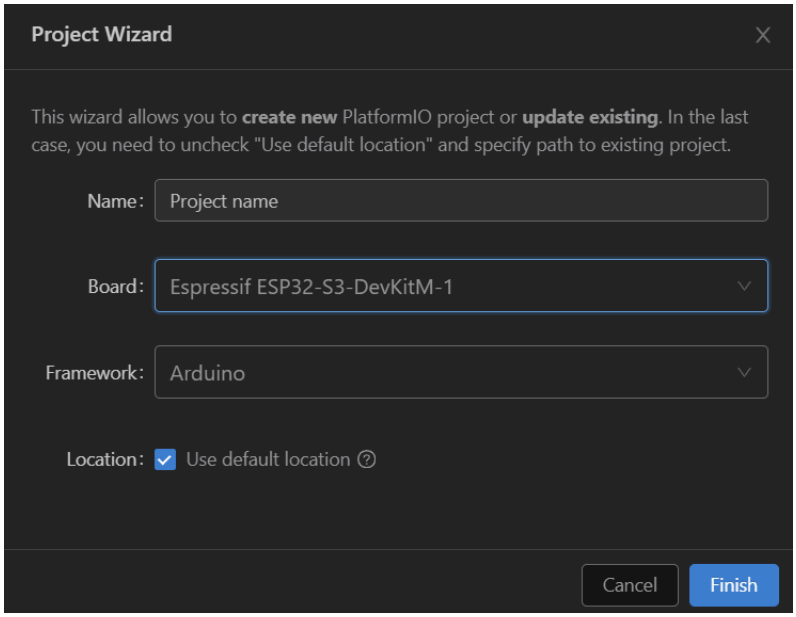


Example 1: Blink

Steps 1 and 2 below follow nearly perfectly this YouTube tutorial: [Get started with PlatformIO and the ESP 32](), with the slight change of the type of board used (see point 2. a.)

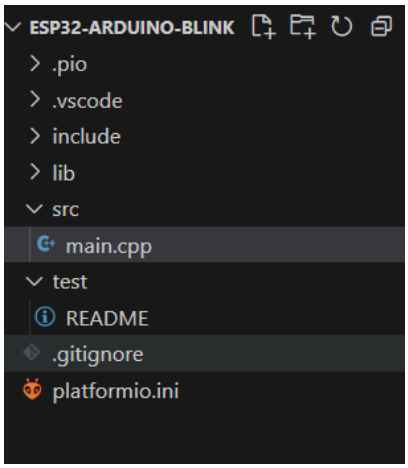
1. Install VSCode and PlatformIO
2. Create a new ESP32-S3 Project
    a. Be sure to create an "Espressif ESP32-S3-DevKitM-1" project (NOTE: when naming files as well as when selecting file locations be sure to have no white space in

the file path anywhere. White space can lead to errors later in the compilation process)



b. Wait for Visual Studio to download and install the ESP32 libraries and create the project.

3. From the left hand file explorer open main.cpp

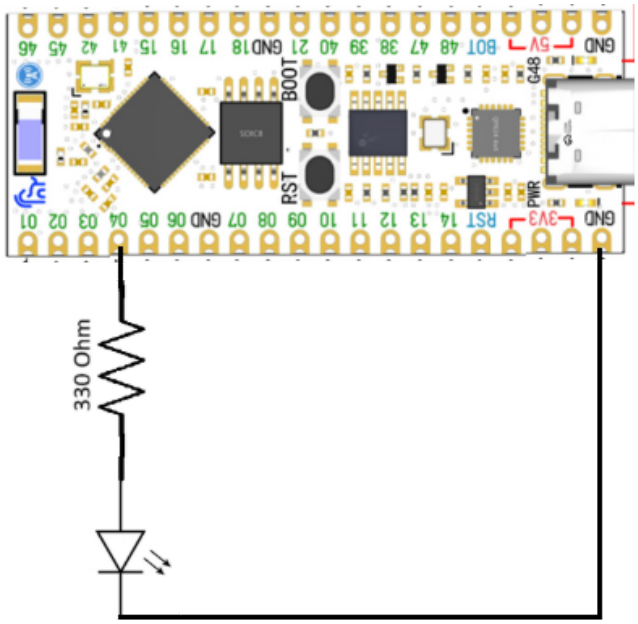


4. Copy and paste the following code int main.cpp so that it reads:

a.
```
// This code will "blink" pin 4 of the esp32 pico for one second every second.
```

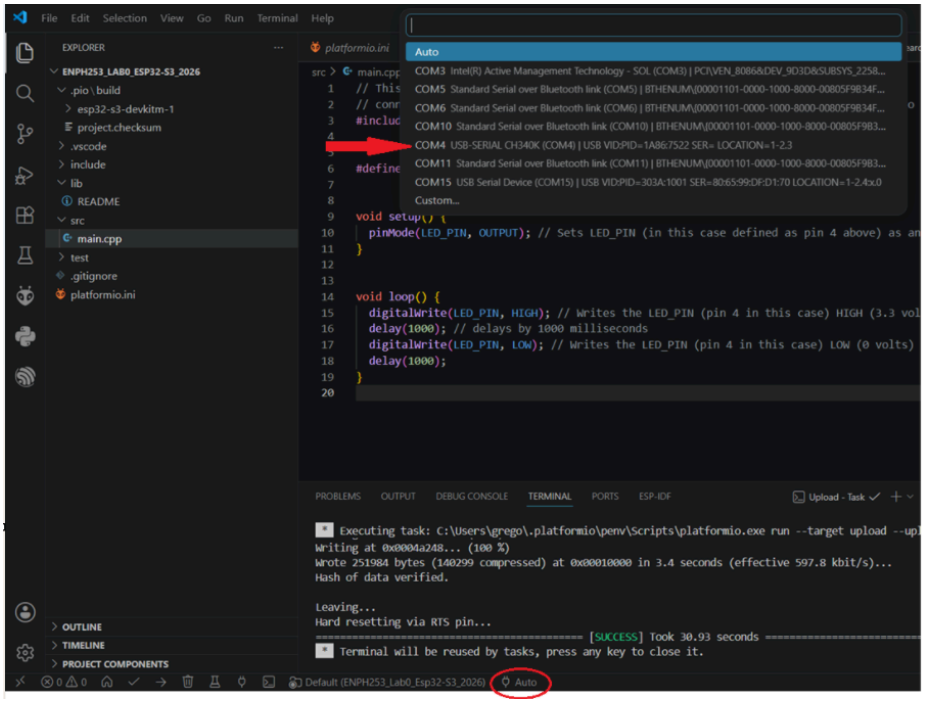
```
// connecting a series resistor (such as a 330 ohm) as well as an LED from this pin to ground will blink the LED.
#include <Arduino.h>

#define LED_PIN 4

void setup() {
  pinMode(LED_PIN, OUTPUT); // Sets LED_PIN (in this case defined as pin 4 above) as an output pin
}

void loop() {
  digitalWrite(LED_PIN, HIGH); // Writes the LED_PIN (pin 4 in this case) HIGH (3.3 volts to the pin which in this case will be powering an LED)
  delay(1000); // delays by 1000 milliseconds
  digitalWrite(LED_PIN, LOW); // Writes the LED_PIN (pin 4 in this case) LOW (0 volts)
  delay(1000);
}
```

5. Now before you upload this code you need to setup your hardware (ESP32, LED, 330 Ohm resistor, breadboard, and some wires)
   a. Connect your ESP32 to your breadboard.
   b. Connect a 330 ohm resistor from pin 4 of the ESPE32 to the anode of an LED (positive terminal, longer leg)
   c. Connect the cathode of the LED (negative terminal, shorter leg) to the GND pin of the ESP32.
   d. Diagram:



6. Now connect your ESP32 to your computer and upload the blink code you copied in the earlier step 4. (you may need to change the upload port from “Auto” to the port on your

computer that has a “ch340k” UART bridge detected, for me in the below diagram it was com 4)



7. The LED should blink for one second every second

Troubleshooting

Linux users

The /dev/ttyUSB0 is under the dialout group and cannot be accessed by all users. Make sure to add your username to the group to allow access to /dev/ttyUSB0

```
$ sudo usermod -a -G dialout $USER
```

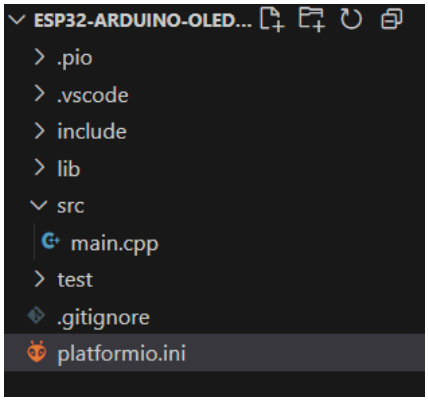
You will then need to log out and log back in.

Example 2: OLED “Hello World”

1. Create a new ESP32 pico project (similar to example 1 above) and call it ESP32_OLED

   NOTE: Do not modify your Blink project. You can use it as the simplest code you upload to test whether an ESP32 board is functional.

2. From the left hand file explorer open platformio.ini



3. Replace the contents of platformio.ini with the following code: (this will allow use of the ssd1306 OLED display, and the monitor speed setting is in case you want to print to the serial monitor on your computer)

```
; PlatformIO Project Configuration File
;
;   Build options: build flags, source filter
;   Upload options: custom upload port, speed and extra flags
;   Library options: dependencies, extra library storages
;   Advanced options: extra scripting
;
; Please visit documentation for the other options and examples
; https://docs.platformio.org/page/projectconf.html

[env:esp32-s3-devkitm-1]
platform = espressif32
board = esp32-s3-devkitm-1
framework = arduino
monitor_speed = 115200
lib_deps =
    Adafruit SSD1306
```

4. Open main.cpp and replace the code with the following:

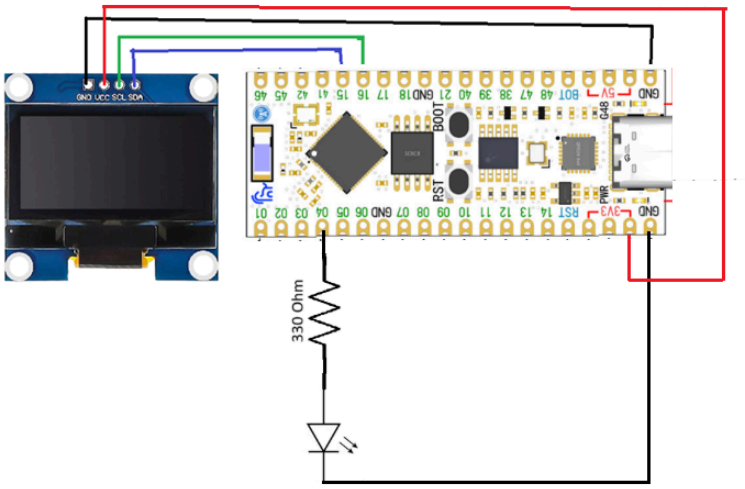
```
#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>


#define SCREEN_WIDTH 128 // OLED display width, in pixels
#define SCREEN_HEIGHT 64 // OLED display height, in pixels

#define I2C_SDA 15
#define I2C_SCL 16
```

```
bool success = Wire.begin(I2C_SDA, I2C_SCL); // sets the I2C pins to the specified values

// Declaration for an SSD1306 display connected to I2C (SDA, SCL pins)
#define OLED_RESET     -1 // Reset pin # (or -1 if sharing Arduino reset pin)
Adafruit_SSD1306 display_handler(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);


void OledSetup(){
  display_handler.begin(SSD1306_SWITCHCAPVCC, 0x3C); // Displays Adafruit logo by default. call clearDisplay immediately if you don't want this.
  display_handler.display();
  delay(2000);

  display_handler.clearDisplay();


  display_handler.setTextSize(1);
  display_handler.setTextColor(SSD1306_WHITE);
  display_handler.setCursor(0,0); // set the cursor start location
  display_handler.println("Hello World");// Displays "Hello world!" on the screen
  display_handler.display();
}


void setup() {
  OledSetup();
}


void loop() {


}
```

5. Now before you upload code you need to setup your hardware (Same as example 1 but additionally an OLED display)
   You don't need to remove any of the example 1 hardware, just add an oled display onto your breadboard somewhere beside your ESP32
   Connect the OLEDs:
   i. GND to ESP32 GND
   ii. VDD to ESP32 3V3
   iii. SCK (sometimes also called SCL) to ESP32 GPIO 16

iv. SDA to ESP32 GPIO 15

Example wiring (if left example 1 wiring):



6. Now after you upload the new code, "Hello World" should be displayed on the first line of the OLED screen.

Analyse previous years' robot designs

1) Analyse at least three robots with the following assessment criteria in mind. On a blank piece of paper, make a simple table with a column for each robot and a row for each of the below points. In each cell, provide a score of 1 (worst) to 10 (best), and a few comments to support each assessment.

Mechanical design considerations:

- **Rigidity and stiffness** of the chassis and major components: are they rigid and strong, or loose and floppy? Can you rely on them as a solid base on which to mount/align sensors and actuators, or will their floppiness lead to variation in sensor readings and difficulty in repeatable operation from run-to-run? Are they robust enough to withstand use and abuse, or are they really delicate? Could this robot be picked up by a non-team member without breaking? Are key components mounted sensibly (bolts, standoffs, 3D-printed fixtures) or are they glued/taped in place? How might this impact robustness and repeatable operation?
- **Quality of moving mechanisms:** for any given moving mechanism (a rotating arm, a linear actuator, etc.), how well does the design implement the desired degree of freedom while eliminating/suppressing undesired degrees of freedom? For example, does the arm rotate smoothly and easily about its primary axis without flopping from side to side?
- **Modularity:** Does it look like it was designed and built with some sense of structure and organization, and can module A be maintained/upgraded without a complete rebuild of the chassis? Or is everything connected in one big organic mess?
- **Strategic implementation:** How well does the design and layout implement the strategy? This is a tough one as you don't know what the designers were thinking, but consider the placement of motors, sensors, switches, range of motion, item storage, layout of subsystems, etc. Does it make sense or not? Why?
- **Elegance:** How "elegant" is the design? Does it look like it was designed with a clear and strategic intention, or does it seem to have "evolved" haphazardly from some primordial soup of 253 components, with lots of ad-hoc and last minute mods? Does it

look like the team iterated and refined the design, possibly rebuilding things a couple of times, or have they just piled new features on top of the old?

- **General mechanical build quality:** If this were a car, would you describe it as well made by someone who knew what they were doing, and did it with care, or the opposite? Why? What gave you your impressions?

High level electrical design (layout) considerations:

- **Layout and accessibility:** How easy or hard would it be to troubleshoot, maintain, and upgrade the electrical systems? Are critical circuits/items (H-bridge, JTAG) accessible for probing, or is everything buried deep in the chassis? If the layout is poor, how difficult would it be to improve? Would it require an entire redesign, or a quick mod? Another way to think about this: is electrical layout something that needs to be thought of early, or can it be sorted out at the end?
- **Cleanliness and quality:** Does the electrical system look clean, organized, and systematic, or is it a rat's nest? How are cables routed? What connectors are used, where, and why? Can circuits be removed for detailed troubleshooting? Are key circuits/connections labelled?
- **Robustness:** How are circuit boards and sensors mounted and how are cables routed and "strain-relieved" (held in place in such a way that force is not transferred to the connectors/boards, leading to stress and failure)? Think again about modularity and needing to maintain or replace sub-systems.

A final note about thinking like a "scientific engineer":

- A well-engineered system operates repeatably and predictably and should not introduce additional "variability". In the robots you review, how could poor quality in the design and execution of the build result in variability in the robot behavior? Floppy mounting of sensors is one example. If your tape sensor is flopping around, it will be very, very hard to get your robot to operate reliably. And no, you can't "fix it in software".